

Lucid

Intelligent Cognitive Alarm Platform
Frontend Development Report

A puzzle-based alarm app built using Flutter

Table of Contents

1. Project Description	1
2. Tools and Technologies Used	2
3. Environment Setup	3
4. Application Screens	4-5
5. Features Implemented	6
6. Proposed System / Architecture	7
7. Development Process and Challenges	8
8. Known Limitations	9
9. Conclusion	9

1. Project Description

Lucid is a smart alarm app. Normal alarms are easy to turn off without really waking up — you just tap once and go back to sleep. Lucid fixes this problem.

- When the alarm rings, it does not stop with a simple tap.
- The user must solve a small puzzle first — either a math question or a memory game.
- Only after solving the puzzle correctly does the alarm stop.
- This forces the brain to become active, so the person is less likely to fall back asleep.

This report explains the **frontend part** of the project — meaning the actual app screens and features that the user sees and interacts with. It was built using Flutter.

Important: Only the frontend (the app itself) has been built so far. There is no backend/server yet, so login/signup in the app is just a demo version for now — it is not connected to any real database.

2. Tools and Technologies Used

Here are the main tools used to build this app, explained in simple terms:

Tool	What it is used for
Flutter	The main framework used to build the app. Lets one app run on Android, iPhone, and desktop.
Dart	The programming language used inside Flutter to write the app's logic.
Provider	Helps manage and update information across the app (like alarms, login status) automatically.
Google Fonts	Used to add clean, modern-looking text styles/fonts to the app.
Audioplayers	A package that plays the alarm sound on loop until the puzzle is solved.
Android Studio	The software used to write, run, and test the app.
Android Emulator	A virtual Android phone on the computer, used to test the app without needing a real phone.

2.1 Planned for later (not built yet)

These are things planned for the future backend part of the project:

- A server (backend) to store users, alarms, and data permanently.
- A real database to save information instead of just storing it temporarily in the app.
- AI/Machine learning to make the puzzle difficulty smarter over time (not present yet — see Section 5.3).

3. Environment Setup

This section explains the setup used to build and run the app.

- **Computer:** Windows laptop.
- **Software used to write code:** Android Studio.
- **Testing method:** Android Emulator (a virtual phone on the computer) — specifically a Pixel 4 phone running Android 15.
- **Flutter version:** 3.29.2
- **Project files:** organized in a clean folder structure so it is easy to add the backend later without rewriting the app.

3.1 Problems faced while setting up

While setting up the emulator, the app kept failing to build with an "Access is denied" error. Here is what was done to fix it, step by step:

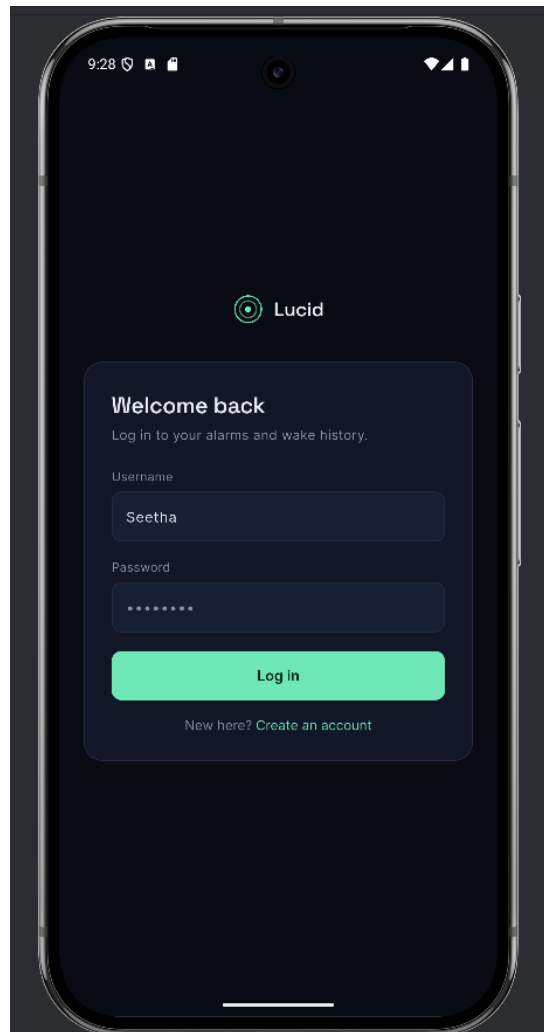
- First checked if Flutter itself was working properly — it was.
- Ran the exact same build command manually (outside of Android Studio) — it worked fine.
- This showed the problem was not with Flutter or the code, but with something blocking the build process only when run through Android Studio.
- Also found and fixed a separate, smaller issue — a missing/corrupted Android NDK (a required tool for Android builds) version.
- Finally found the real cause: the antivirus software installed on the laptop (360 Total Security) was blocking the build process.
- Fixed by pausing the antivirus's real-time protection while building the app.

4. Application Screens

Below are screenshots of the app running on the Android emulator.

Figure 1: Login screen

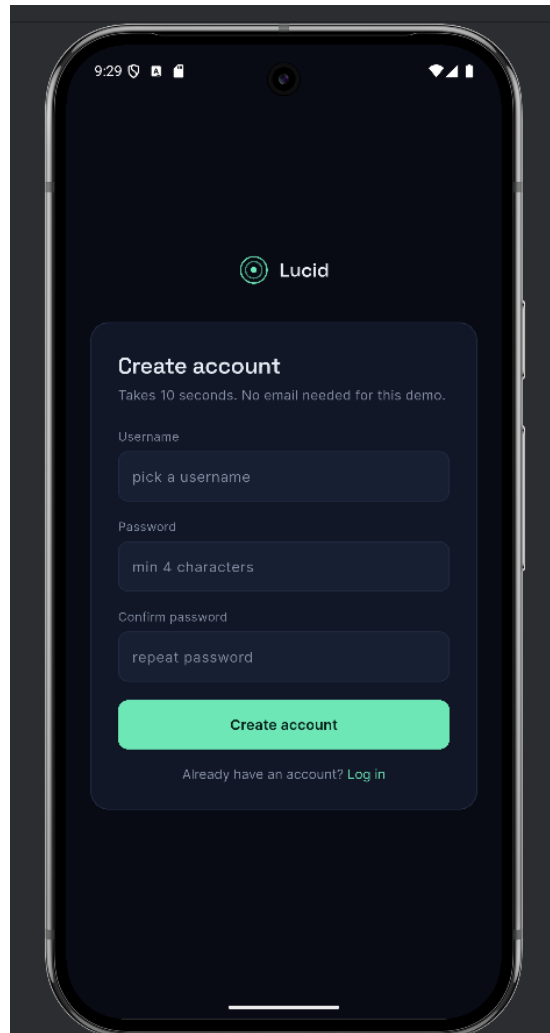
Screenshot_2026-07-08_212916.png



This is the first screen a returning user sees. They enter their username and password to log in.

Figure 2: Create account screen

Screenshot_2026-07-08_213010.png

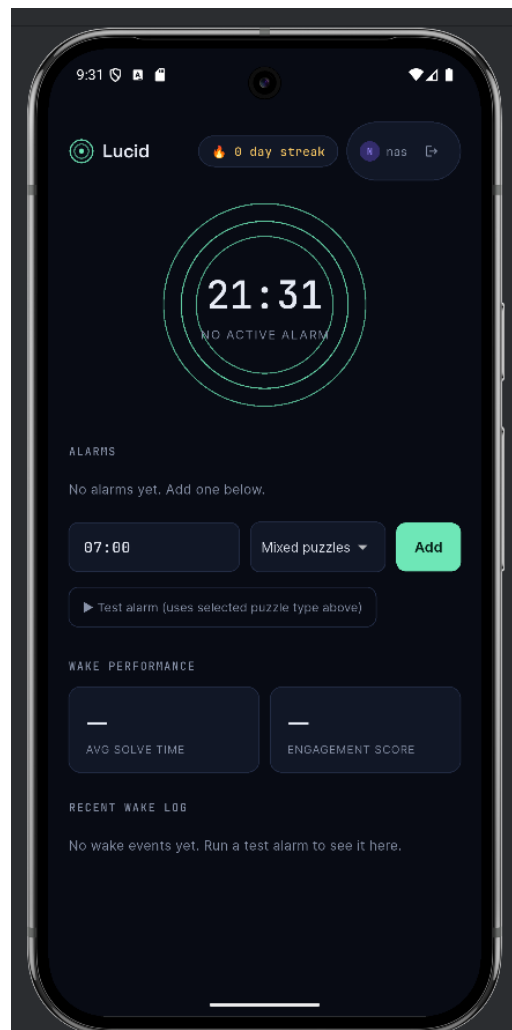


The screenshot shows a mobile phone screen with a dark background. At the top, the status bar shows the time 9:29 and various icons. Below the status bar, the Lucid logo is centered. The main content is a white card with the title 'Create account' and a subtitle 'Takes 10 seconds. No email needed for this demo.' The card contains three input fields: 'Username' with a placeholder 'pick a username', 'Password' with a placeholder 'min 4 characters', and 'Confirm password' with a placeholder 'repeat password'. Below these fields is a prominent green button labeled 'Create account'. At the bottom of the card, there is a link that says 'Already have an account? Log in'.

This screen is for new users. They pick a username, set a password, and confirm it to create an account.

Figure 3: Home screen (dashboard)

Screenshot_2026-07-08_213116.png



This is the main screen after logging in. It shows:

- A clock in the middle with a glowing ring animation.
- The user's current streak (number of days they woke up successfully).
- A list of alarms the user has set.
- A place to add a new alarm by choosing a time and puzzle type.
- A button to test the alarm immediately, without waiting.
- Stats showing average solving time and engagement score.
- A log of recent wake-up events.

Figure 4: Alarm ringing screen (puzzle screen)

Screenshot_2026-07-08_213245.png



This screen appears when the alarm rings. In this example, it shows a math question. The user must type the correct answer to stop the alarm. This screen cannot be closed by pressing the phone's back button — the only way to stop the alarm is to solve the puzzle.

5. Features Implemented

5.1 Login and Sign up

- Users can create an account with a username and password.
- Users can log in with their existing account.
- Each user's alarms and data are kept separate from other users.
- A logout button is available on the home screen.

5.2 Setting Alarms

- Users can add an alarm by choosing a time and a puzzle type (Math / Memory / Mixed).
- Alarms can be turned on or off using a simple switch.
- The app checks the time every second and automatically rings the alarm when it matches.
- A "Test alarm" button lets the user try the alarm immediately, without waiting.

5.3 The Puzzle (Core Feature)

- **Math puzzles:** Simple addition at first, getting harder (subtraction, multiplication, and combined questions) as the difficulty increases.
- **Memory puzzles:** The app lights up a pattern on a 3x3 grid, and the user must tap the same pattern back in the correct order.
- If the user answers wrong, the puzzle becomes slightly harder and a new question appears — it always stays the same type (so "Memory only" always gives memory puzzles, never math).
- **Note:** The way the puzzle gets harder is a simple rule (get one level harder after each wrong answer) — it is **not** AI or machine learning. This was confirmed during development to avoid any confusion. A smarter, AI-based version may be added in the future.
- The puzzle screen cannot be skipped or closed using the back button — it can only be closed by solving it.

5.4 Alarm Sound

- The alarm plays a real sound on loop until the puzzle is solved.
- If no sound file is added, it automatically plays a backup beep sound instead, so it is never silent.

5.5 Tracking Wake-up Performance

- Every time an alarm is solved, the app records: what time it rang, how long it took to solve, how many wrong attempts were made, and a score out of 100 (called the engagement score).
- A streak counter goes up by one every time the user successfully wakes up and solves the puzzle.
- The home screen shows the average solving time, average score, and a list of recent wake-ups.

5.6 App Design

- A dark theme with a calm green color for the normal screens.
- The color changes to purple/violet when the alarm is ringing, to grab attention.
- A glowing, pulsing ring animation around the clock, to give the app a calm and modern feel.
- Clean, easy-to-read fonts used throughout the app.

6. Proposed System / Architecture

Only the frontend (app) has been built so far. But it was designed keeping the bigger picture in mind, so a backend (server) can be added later without rebuilding the whole app. Here is how the complete system is planned to work, in simple steps:

- **Step 1:** The user opens the app (this is the frontend, already built).
- **Step 2:** The app will send requests to a server (this is the backend, not built yet).
- **Step 3:** The server will handle things like checking logins, saving alarms, and checking puzzle answers.
- **Step 4:** The server will store all this information safely in a database, so it is not lost when the app is closed.

A basic database design (a list of what information needs to be stored, like users, alarms, and puzzle results) has already been planned in advance, so building the server later will be easier.

6.1 How the app's code is organized

The app's code is split into clear, organized parts:

main.dart — the starting point of the app

theme folder — colors and fonts used across the app

models folder — defines what an Alarm, a Wake Log, and a User look like

state folder — handles login, alarms, and puzzle logic

screens folder — the login screen, home screen, and alarm-ringing screen

widgets folder — reusable pieces like the clock animation and alarm list item

7. Development Process and Challenges

The app was built step by step, in this order:

- First, planned the overall idea and how the different parts of the full project would connect.
- Designed a basic database plan for future use.
- Built a quick test version of the app using simple web code (HTML/CSS/JavaScript) to check if the alarm-and-puzzle idea actually worked well.
- Fixed small bugs in that test version — like a puzzle type not matching what was selected, and text being hard to read.
- Added a basic login/signup screen to the test version.
- Rebuilt the entire app properly using Flutter, in an organized way.
- Added a real alarm sound that loops until the puzzle is solved.

7.1 Problems faced and how they were solved

A large part of the work involved fixing a build error that kept happening only on the Android emulator (not on Windows). Steps taken to solve it:

- Checked that Flutter was installed correctly — confirmed it was working fine.
- Tried running the build command directly (without Android Studio) — this worked, which helped narrow down the problem.
- Along the way, also fixed an unrelated issue with a missing Android tool (NDK) that needed to be reinstalled.
- Eventually found that the laptop's antivirus software was blocking the build process.
- Fixed it by pausing the antivirus temporarily while building the app.

8. Known Limitations

Since only the frontend has been built so far, here is what is missing:

- **No server/backend yet** — all data is stored only in the app's memory and is lost when the app is restarted.
- **Login is not real/secure yet** — it is a demo version just to show how it will work.
- **Alarms only work while the app is open** — there is no background alarm feature yet, so the phone must have the app open for the alarm to ring.
- **Puzzle answers are checked only inside the app** — a server would be needed later to check answers safely and prevent cheating.
- **No AI/Machine Learning is used** — the puzzle only gets harder using a simple fixed rule, not a smart/learning system.

9. Conclusion

The Lucid app successfully shows how a puzzle-based alarm can work. Users can create an account, log in, set alarms with a chosen puzzle type, and must solve a math or memory puzzle to stop the alarm — with the puzzle getting harder if they get it wrong. The app also tracks streaks and wake-up performance, all inside a clean, modern-looking design.

This project was built with future growth in mind. The next steps would be building a real server, adding secure login, making alarms work in the background, and possibly adding smart (AI-based) difficulty adjustment later on.